QuantRule - Architecture Description

2026-11-12

1. Style

Service-Based architecture: six coarse services, one database instance with one schema per service, one object store, one broker, one image. The data plane is a synchronous pipeline; the control plane is asynchronous; research is a separate, isolated plane.

Chosen over microservices (saga and mesh cost for one operator) and over a monolith (cannot detach a run from the client or use a second machine).

2. Partitioning

Domain-driven, by capability, not by layer:

Service	Capability	Kata
`gateway`	HTTP boundary, static client, profile, upload	R6, R8
`data`	Ingestion, content identity, ladder, catalog	R1, R2
`model`	Readings, states, events, progression	R2
`backtest`	Orchestration, strategy, execution, portfolio	R3, R4
`evidence`	Provenance, digest, run records, memory	R5
`simulation`	Windows, marks, oracle, review, check	R7

3. Components and boundaries

```
browser / shell / desktop shell
  -> gateway (HTTP, quantrule.api/v1)
       -> postgres
       -> evidence -> object store
  -> broker
       -> backtest -> object store
            -> model
            -> data
  -> simulation (research plane: no queue, no run id)
```

- Stages exchange content references, never payload sets.
- Database, object store, and queue sit behind traits; a service calls the trait.
- The research plane can read a recorded bundle and cannot write a run artifact.

Code: `crates/quantrule-{api,server,app,data,model,backtest,shell}` hold the contract, the services, the use cases, and the engines; `apps/web` and `apps/desktop-shell` hold the two human clients.

4. Interfaces

Interface	Shape	Version
Client API	HTTP/JSON	`quantrule.api/v1`
Control plane	Message envelope	`quantrule.queue/v1`
Run manifest	JSON	`quantrule.run/v1`
Evidence bundle	Files plus provenance	`quantrule.bundle/v1`
Shell	Command contract	in-repo

No interface accepts a filesystem path. No interface returns a label: the gateway forwards numeric codes and each client maps them.

5. Constraints

- Rust for every computation; Svelte for the client; no Python in the image.
- One image, six binaries, one digest.
- One principal; profiles organise records and are not a security boundary.
- Determinism is compared by a canonical digest, not by a whole-bundle hash.

6. Decisions

Twelve recorded decisions, each with context, alternatives, consequences, a revisit trigger, and verification:

1. Service-Based architecture with six services.
2. Control plane separate from the data plane.
3. One principal, no accounts.
4. Content-addressed exchange; one image digest.
5. Svelte client; the desktop shell owns no domain.
6. A run is the full pipeline; inspection is read-only.
7. Canonical digest, and the limit of the determinism claim.
8. The static download page is retired.
9. A simulation service owns the research plane.
10. Python is confined to repository tooling; the image has none.
11. Profiles are organisational, not a security boundary.
12. Desktop: thin shell now, embedded later, in-process never.

7. Changeability evidence

Two changes carried through the architecture:

Change	Touched	Cost	What was hard
Add the simulation service	2 decisions, 9 work items; no engine change	one documentation pass	keeping hindsight out of the signal path
Add profiles	one column across four schemas, one header, one client store	small	every query must be scoped; a missed scope is a silent leak between research lines

A rule change (R3) touches one crate and at most three modules: the strategy vocabulary, its config parser, and its test fixture. A new reading (R2) touches one reading module plus the registry, the config parser, the writer, and one golden fixture - the widest ripple in the system, and it is bounded.

Expensive to change: the canonical digest definition (invalidates recorded evidence), the run identity inputs (invalidates deduplication), and the service count (a new decision and a new deployment unit).

8. Consistency

Kata R1-R8 -> eight characteristics -> six services -> twelve decisions -> the issue tree. One increment is described throughout: the distributed platform that makes one rule produce one reproducible bundle, and one research question produce one checked hypothesis.